3. Longest Substring Without Repeating Characters Medium

Acceptance: 38.3% | LeetCode

Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:

```
**Input:** s = "abcabcbb"
**Output:** 3
**Explanation:** The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
```

Example 2:

```
**Input:** s = "bbbbb"
**Output:** 1
**Explanation:** The answer is "b", with the length of 1.
```

Example 3:

```
**Input:** s = "pwwkew"
**Output:** 3
**Explanation:** The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- `0 <= s.length <= 5 * 104`
- `s` consists of English letters, digits, symbols and spaces.

Claude Opus Solution

Longest Substring Without Repeating Characters

1. Intuition

The key insight is the **Sliding Window** pattern. We maintain a "window" (a contiguous substring) that contains **no duplicate characters**. As we scan through the string:

- We **expand** the window by moving the right pointer forward.
- When we hit a duplicate character, we **shrink** the window from the left until the duplicate is removed.

The critical optimization: instead of moving the left pointer one step at a time, we can **jump** it directly past the last occurrence of the duplicate character using a hash map. This avoids redundant work.

Why this works: A substring without repeats is valid only if every character appears once. By tracking the *last seen index* of each character, we know exactly where to "cut" the window's left boundary when a repeat appears.

---

2. Approach (Pseudocode)

```
Initialize:
    last_seen = {}        # maps character -> its most recent index
    left = 0              # left boundary of the window
    max_length = 0

For right from 0 to len(s) - 1:
    char = s[right]

    # If char was seen AND is inside the current window,
    # move left pointer just past its previous occurrence.
    if char in last_seen and last_seen[char] >= left:
        left = last_seen[char] + 1

    # Record/update the latest index of this character.
    last_seen[char] = right

    # Current window size = right - left + 1; track the maximum.
    max_length = max(max_length, right - left + 1)

Return max_length
```

---

3. Python Solution

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # Maps each character to the LAST index where we saw it.
        last_seen = {}

        left = 0          # Left boundary of our sliding window.
        max_length = 0    # Best (longest) window length found so far.

        # 'right' expands the window one character at a time.
        for right, char in enumerate(s):

            # Check if 'char' is a duplicate that lies INSIDE the current window.
            # Condition `last_seen[char] >= left` is essential:
            # a character may exist in the map but be OUTSIDE the window
            # (i.e., already passed by 'left'), in which case it's not a conflict.
            if char in last_seen and last_seen[char] >= left:
                # Jump 'left' to just after the previous occurrence,
                # instantly removing the duplicate from the window.
                left = last_seen[char] + 1

            # Update the latest position of this character.
            last_seen[char] = right

            # The window [left, right] is now valid (no duplicates).
            # Its length is (right - left + 1); update the max.
            max_length = max(max_length, right - left + 1)

        return max_length
```

Walkthrough on `"pwwkew"`:

right	char	left before	duplicate?	left after	window	max_length
0	p	0	no	0	"p"	1
1	w	0	no	0	"pw"	2
2	w	0	yes (idx 1)	2	"w"	2
3	k	2	no	2	"wk"	2
4	e	2	no	2	"wke"	3
5	w	2	yes (idx 2)	3	"kew"	3

Result: **3** ✓

---

4. Complexity Analysis

Time Complexity: O(n) - We iterate through the string exactly **once** with the `right` pointer. - The `left` pointer only ever moves forward (it never resets backward), so its total movement across the whole run is also bounded by `n`. - Each hash map lookup/update is O(1) on average. - This is the **optimized** sliding window — note that `solution2` in the reference moves `left` one step at a time, which can revisit characters, though it's still amortized O(n).

Space Complexity: O(min(n, m)) - `m` is the size of the character set (e.g., 128 for ASCII, 26 for lowercase letters). - The hash map stores at most one entry per distinct character. - So space is bounded by `min(string length, alphabet size)`.

---

5. Alternative Approaches

Approach	Idea	Time	Space	Notes
Brute Force	Check every substring for uniqueness	$O(n^3)$	O(min(n,m))	Too slow; checks all $O(n^2)$ substrings, each O(n) to validate
Sliding Window + Set	Expand right; when duplicate, shrink left one-by-one (reference `solution2`)	O(2n)	O(min(n,m))	Each char visited at most twice; simpler but slower constant
Sliding Window + Map (above)	Jump `left` directly past duplicates	O(n)	O(min(n,m))	Optimal; `left` moves only forward

Approach	Idea	Time	Space	Notes
Fixed-size array	Replace hash map with `int[128]` array	O(n)	O(1)	Same logic; faster lookups since ASCII is bounded

Bonus: Array-based version (fastest in practice)

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # Since input is ASCII, use a fixed array instead of a dict.
        # index = ASCII code, value = last seen position (+1 for "unset").
        last_seen = [-1] * 128
        left = max_length = 0

        for right, char in enumerate(s):
            code = ord(char)
            # Move left past the previous occurrence if inside window.
            left = max(left, last_seen[code] + 1)
            last_seen[code] = right
            max_length = max(max_length, right - left + 1)

        return max_length
```

This uses `left = max(left, last_seen[code] + 1)` to elegantly handle the "inside window" check without an explicit `if`, and replaces the dictionary with a constant-size array for **O(1) true space** and faster constant-time access.

Codex Solution

1. Intuition

We need the longest **contiguous substring** with no repeated characters.

This is a classic **sliding window** problem.

Think of maintaining a window:

```
s[left ... right]
```

such that the current window always contains **unique characters**.

As we scan from left to right:

- Expand the window by moving `right`
- If we see a repeated character, move `left` forward until the duplicate is removed
- Track the maximum valid window length seen so far

A more optimized version uses a hash map storing the **last seen index** of every character.
When we see a duplicate, we can jump `left` directly instead of moving one step at a time.

---

2. Approach

Key idea

Store the last index where each character appeared.

When processing character `s[right]`:

- If it was seen before **inside the current window**, move `left` to one position after its previous occurrence.
- Update the character's latest index.
- Update the maximum window length.

Pseudocode

```
function lengthOfLongestSubstring(s):
    last_seen = empty hash map
    left = 0
    answer = 0

    for right from 0 to len(s) - 1:
        char = s[right]
```

```
        if char is in last_seen and last_seen[char] >= left:
            left = last_seen[char] + 1

        last_seen[char] = right

        current_length = right - left + 1
        answer = max(answer, current_length)

    return answer
```

3. Python Solution

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # Dictionary mapping each character to its most recent index
        last_seen = {}

        # Left boundary of the current sliding window
        left = 0

        # Stores the length of the longest valid substring found so far
        max_length = 0

        # right is the right boundary of the current sliding window
        for right, char in enumerate(s):

            # If char was seen before and its previous index is inside
            # the current window, then we have a duplicate.
            #
            # Example:
            # s = "abba"
            # When right = 2, char = 'b'
            # previous 'b' is at index 1, which is inside the window.
            #
            # So we move left to index 2.
            if char in last_seen and last_seen[char] >= left:
                left = last_seen[char] + 1

            # Update the most recent position of this character
            last_seen[char] = right

            # Current window is s[left : right + 1]
            current_length = right - left + 1

            # Update the best answer found so far
            max_length = max(max_length, current_length)

        return max_length
```

4. Complexity Analysis

Let `n` be the length of the string.

Time Complexity: `O(n)`

Each character is processed once by the `right` pointer.

The `left` pointer only moves forward and never moves backward.

Dictionary lookups and updates are average `O(1)`.

Therefore, total time complexity is:

```
O(n)
```

Space Complexity: `O(k)`

The hash map stores at most one entry per distinct character in the string.

Let `k` be the number of distinct characters.

```
O(k)
```

Since the input consists of English letters, digits, symbols, and spaces, `k` is bounded by the character set size.

In the worst case, if all characters are unique:

```
O(n)
```

---

5. Alternative Approaches

1. Sliding Window with a Set

Instead of storing last indices, we can maintain a set of characters in the current window.

If we find a duplicate, repeatedly remove characters from the left until the duplicate is gone.

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        seen = set()
        left = 0
        max_length = 0

        for right, char in enumerate(s):
            # Shrink the window until char is no longer duplicated
            while char in seen:
                seen.remove(s[left])
                left += 1

            # Add current character to the valid window
            seen.add(char)

            # Update answer
            max_length = max(max_length, right - left + 1)

        return max_length
```

Time complexity is still `O(n)` because each character is added and removed at most once.

---

2. Brute Force

Check every possible substring and test whether it contains duplicates.

This is much slower.

```
Generate all substrings: O(n^2)
Check uniqueness: up to O(n)
Total: O(n^3)
```

With optimization using a set while expanding each start position, it can be reduced to `O(n^2)`.

However, the sliding window solution is preferred because it runs in `O(n)`.